

Challenges of shared decision-making: A multiple case study of agile software development

Nils Brede Moe^a, Aybüke Aurum^{b,*}, Tore Dybå^a

^a SINTEF, NO-7465 Trondheim, Norway

^b School of Information Systems, Technology and Management, University of New South Wales, Sydney, Australia

ARTICLE INFO

Article history:

Available online 28 November 2011

Keywords:

Agile software development
Scrum
Decision-making
Self-management
Alignment
Teamwork

ABSTRACT

Context: Agile software development changes the nature of collaboration, coordination, and communication in software projects.

Objective: Our objective was to understand the challenges of shared decision-making in agile software development teams.

Method: We designed a multiple case study consisting of four projects in two software product companies that recently adopted Scrum. We collected data in semi-structured interviews, through participant observations, and from process artifacts.

Results: We identified three main challenges to shared decision-making in agile software development: alignment of strategic product plans with iteration plans, allocation of development resources, and performing development and maintenance tasks in teams.

Conclusion: Agile software development requires alignment of decisions on the strategic, tactical, and operational levels in order to overcome these challenges. Agile development also requires a transition from specialized skills to redundancy of functions and from rational to naturalistic decision-making. This takes time; the case companies needed from one to two years to change from traditional, hierarchical decision-making to shared decision-making in software development projects.

Crown Copyright © 2011 Published by Elsevier B.V. All rights reserved.

1. Introduction

Agile software development (ASD) involves a radically new approach to decision-making in software projects. ASD teams deliver working software in short iterations [11,31], which results in more frequent, short-term decisions, compared with a traditional software development approach. Agile development requires the team to adopt a collaborative, but speedy decision-making process [7]. Moreover, developers are organized in self-managing teams [36], and they are encouraged to become involved in project management decisions that may fall outside of their traditional skill areas. The client should be consulted regularly throughout the project, and involved in most of the project management decisions [5].

When adopting agile methods in an organization based on traditional, plan-driven development models, the locus of decision-making moves from the project manager to the software development team, and the decision-making process changes from individual and centralized to shared and decentralized. Thus leadership is shared, and important decisions on what to do and how to do it

are made through an interactive process involving many people who influence each other, not just a single person [31]. Such collaborative decision-making, which involves stakeholders with diverse backgrounds and goals, is more complicated than traditional approaches, where the project manager is responsible for most of the decisions [36]. Therefore to implement ASD successfully it is important to explore and understand the challenges of shared decision-making. This leads to the following research question:

What are the challenges of shared decision-making in agile software development teams?

This paper describes and reflects on the challenges of shared decision-making in four projects in two software companies, which have adopted agile methods. It shows that introducing Scrum requires changes at all levels of the organization: from how resources are allocated, how the organization supports the teams, and how backlogs are aligned with strategic decisions, to power shifts in decision-making, and to the implementation of a shared decision-making process at the operational level. The main contribution of the paper is the deeper insight it offers into the need for reorientation not only by the developers, but also by management, when transitioning from specialized skills to redundancy of functions and from rational to naturalistic decision-making.

* Corresponding author. Fax: +61 296624061.

E-mail addresses: nils.b.moe@sintef.no (N.B. Moe), aybuke@unsw.edu.au (A. Aurum), tore.dyba@sintef.no (T. Dybå).

The remainder of the paper is organized as follows. In Section 2, we present background information on decision-making in an agile context. In Section 3, we describe our research method in detail. In Section 4, we present the results of a multiple case study into the challenges associated with aligning strategic product plans with iteration plans, allocating resources, and performing tasks. We discuss our findings in Section 5. Section 6 concludes and presents suggestions for future research.

2. Decision-making in an agile context

The introduction of agile development does not change the fundamental knowledge required to develop software, but it does change the nature of collaboration, coordination, and communication in software projects. This may alleviate or exacerbate development problems [11]. Hence, determining the influence of decision-making in this context was the motivation for this study.

Several researchers consider the alignment of managerial decisions in organizations, and stress the importance of ensuring that product and project level decisions are well aligned with the organizational business strategy [37,3]. Since moving from a plan-driven approach to an agile approach is a part of business strategy, it is important that software practitioners understand their role in this strategy. A failure of such understanding may easily lead to the failure of the agile strategy itself. Hence, this research is based on the assumption that the alignment of product and project decisions with business strategy is important for the successful application of an agile approach in software development.

In this section we present background information on decision-making, specifically in an agile context. First we present the different levels of decision-making: strategic, tactical, and operational decisions. Second we introduce rational decision-making (the traditional way of understanding how decisions are made), and bounded rational decision-making (which is useful for understanding decision-making in non-routine activities). Third we present naturalistic decision-making (which is useful for understanding decision-making in a real world context). Finally we describe the relationship between self-managing teams and decision-making processes in ASD.

2.1. Understanding decision-making process

There are three general levels of decision-making in organizations depending on the purpose of the management activity: strategic decisions, tactical decisions, and operational decisions [2]. The boundaries between these levels are not always distinct. However, they differ from one another in terms of information requirements. *Strategic decisions* are related to organizational goals and objectives. The information concerning such decisions is usually incomplete and the decision-making process may extend over a considerable period of time. *Tactical decisions* are related to identification and use of resources, while *operational decisions* deal with ensuring effectiveness of day-to-day operations within the organization.

Anthony's model assumes a top-down view of the organization, and a hierarchical view of how decisions are made. However it is becoming more common for lower level employees to be actively involved not only in the day-to-day processes, which were once the domain of middle and senior managers, but in all processes of strategic significance [20]. Furthermore, agile puts the traditional hierarchical view on decision-making in question, with its bottom up approach and the importance of understanding decision-making as a shared process.

Software development involves both routine and non-routine activities. Hence, it makes sense to use both rational and bounded rational decision-making theories when explaining decisions in

software development processes. In rational decision-making it is assumed that when people have complete information about a problem, they can identify all possible solutions, and then chose the one that will maximize the outcome of their efforts [26]. This model is useful when explaining routine decisions, and when used in association with other theoretical frameworks or approaches.

When explaining non-routine decisions, where goals are constantly changing, the bounded rationality model [44] is often useful. This model assumes that decision makers' rationality is restricted by the information they have, their cognitive limitations, and the types of problems they face. An uncertain task requires that large amount of information be processed by decision-makers while performing the task [34]. The greater the uncertainty, the more difficult it is to program and routinize an activity by preplanning a response [10]. When uncertainty increases it is more important to control output (e.g. by setting goals and targets) rather than to control behavior (e.g. through rules and programs). This can be achieved by relying on continuous feedback and mutual adjustment [30].

Another relevant theory on decision-making is naturalistic decision-making, which aids in understanding how people make decisions in real-world contexts that are meaningful and familiar to them [12,25]. The advent of naturalistic decision-making shifted the concept of human decision-making from a domain independent general approach to a knowledge-based approach exemplified by decision makers with substantial experience. The decision-making process was expanded to include the stage of perception and recognition of situations as well as generation of appropriate responses, not just a choice from given options [18].

One example of the use of rational and naturalistic decision-making in software development is the study by Zannier et al. [49]. They found that the more structured a design problem was, the more practitioners seemed to rely on rational approaches to solve it. When the design problem was less structured, practitioners relied on naturalistic approaches to solve it. They also found that while software practitioners often rely on either a naturalistic or rational approach as their dominant strategy, they also use aspects of the other approaches depending on the character of the decision.

2.2. Decision-making in agile software development

Moving from plan-driven to agile development implies moving from command and control to shared decision-making, and self-management [31]. In agile software development, developers work in small teams with customers that represent the system's users. The features to be implemented in each development cycle are jointly decided by the customer and the rest of the development team. Collaborative decision making involving stakeholders with diverse backgrounds and goals is thus a characteristic of agile development [36]. Also team members, empowered with more decision-making powers, are not confined to a specialized role. Moving from plan-driven to agile development can also be seen as moving from rational to naturalistic decision-making.

Shared decision-making is an example of the concept of 'barely sufficient' command and control [29]. The shared decision-making approach, which involves stakeholders with diverse backgrounds and goals, is more problematic than the traditional rational or bounded rational approach where the project manager is responsible for most of the decisions [36].

To fully explain decision-making in agile teams it is necessary to understand how the concept of self-management affects the decision-making process. In a self-managing team, team members have responsibility not only for executing their tasks but also for monitoring and managing their own performance [14]. Also, leadership in such teams should be diffused rather than centralized [34]. Shared leadership can be seen as a manifestation of fully developed empowerment in a team [17]. When the team and the team leaders share

leadership, it is transferred to the person with the key knowledge, skills, and abilities related to the specific issues facing the team at any given moment [38]. While the project manager maintains leadership for project management duties, the team members lead and are responsible for decisions when they possess the knowledge that needs to be shared during different phases of the project [15].

Self-management can directly influence team effectiveness since it brings decision-making authority to the level of operational problems and uncertainties, thus increasing the speed and accuracy of problem solving [46]. However, effective self-managing units cannot be created simply by exhorting democratic ideals, by tearing down organizational hierarchies, or by instituting one-person-one-vote decision-making processes [14]. Hackman identified five general conditions that appear to foster and support self-management: (a) clear, engaging direction, (b) an enabling performing unit structure, (c) a supportive organizational context, (d) available, expert coaching, and (e) adequate resources. Adaptability is also important in such teams since operational decisions are made incrementally while important strategic decisions are delayed as much as possible, in order to allow a more flexible response to last minute feedback from the market place [45].

Product and project levels decisions in a software company can be considered at the strategic, tactical, and operational levels [4]. Therefore, in an ASD product company strategic decisions are primarily related to product and release plans, which may require creativity and opportunistic inputs, and should be based on an accurate understanding of the current business process and a detailed knowledge of the software product. ASD tactical decisions involve the project management view, where the aim is to determine the best way to implement ASD strategic decisions, i.e. to allocate the resources. On the other hand, ASD operational decisions are about implementation of product features and the process of assuring that specific tasks are carried out effectively and efficiently.

In ASD the Product owner is responsible for strategic decisions. However, the project team should also be involved, because the team possesses the relevant technical knowledge. A self-managing team is responsible for shared decision-making on the tactical and operational levels, however the Product owner should also be involved because he or she possesses the business perspective.

3. Research methods

Since the goal of this research is to explore and provide insight into the shared decision-making processes in agile software development, it is important to study software development teams in practice. In this study the focal point is Scrum, because Scrum focuses on self-management and on several processes which involve shared decision-making.

Table 1
Product development teams.

	South Company		North Company	
	Project A	Project B	Project C	Project D
Project size	3 years, 9000 h per year	3 years, 9000 h per year	9 months, 5000 h	12 months, 4000 h
Developers	7 (Norway: 4, India: 3)	5 (including 2 consultants)	6	4
Scrum master	One of the developers	One of the developers	From the quality and marketing department	The department head
Product owner	From the sales and business department	Former project manager	Former project manager, in another city	Former project manager, in another city
Scrum introduced	Middle of project	Middle of project	Beginning of project	Beginning of project
Sprint length	3 weeks	2–4 weeks	4 weeks	2–4 weeks
Scrum practices	All. Not retrospective in every sprint	All	All. Not retrospective in every sprint	All. Not retrospective in every sprint
Physical wall with tasks	No	Yes	No	Yes
System developed	Information system for integrity management of pipelines, both off-shore and on-shore	Information system for designing and maintaining off-shore installations	Geographical information system for planning and coordination work	Information system for handling reports from clients

We designed a holistic multiple case study [48] of four projects in two software product companies which recently adopted Scrum. According to Yin, case studies are the preferred research strategy when a “... question is being asked about a contemporary set of events over which the investigator has little or no control.” (ibid, p. 9). We chose two projects which used Scrum from the start, and two projects which introduced Scrum in the middle of the project.

3.1. Case study design

When conducting the multiple case study we followed the five step process proposed by Yin:

1. Case study design: objectives are defined and the case study is planned.
2. Preparation for data collection: procedures and protocols for data collection are defined.
3. Collecting evidence: execution of data collection on the studied case.
4. Analysis of collected data.
5. Reporting.

The first author planned the introduction of Scrum collaboratively with the companies that were to be the subject of this study, making it possible to plan and collect data in the same way across all projects. Case studies can be based on any mix of quantitative and qualitative evidence, and having multiple sources of evidence ensures construct validity and to achieve triangulation. In this study we relied on triangulation of data sources. The first author interviewed all team members and conducted participant observations in all the teams. A few of the interviews at the North Company were conducted by Torgeir Dingsøyr. The first author also collected documents and physical artifacts. Examples of documentation are minutes of meetings, project plans and schedules. Examples of physical artifacts are the Scrum board and story cards, which were photographed. More details on how data was collected can be found in Section 3.3.

The data was analyzed and reported with the two other authors. Analysis and reporting of the data can be found in Section 3.4.

3.2. Study context

This study was conducted in the context of a larger action research program, where several companies introduced elements of agile development in response to identified problems. One of the companies was situated in the south of Norway and the other in the north, therefore we refer to the companies as the “South Company” and the “North Company” (see Table 1). The description of the two companies is somewhat limited due to confidentiality issues.

All developers in the projects participated in two-day Scrum training, which included tailoring the ASD process to the needs of the company. Three of the four Scrum masters attended additional training and became certified Scrum masters. Furthermore, the Product owners had some Scrum training as well. All projects followed all Scrum practices, however retrospectives were not performed at the end of every sprint.

The Scrum teams were given significant authority and responsibility for many aspects of their work, such as planning, scheduling, assigning tasks to members, and making decisions. The Scrum master was in charge of solving problems that could prevent the team from working effectively. He or she did not organize the teams (designers and developers), but let them organize themselves and make decisions concerning what to do. The prime responsibility of the Scrum master was to remove the impediments to the process, conduct daily meetings, making decisions in these meetings, and corroborating these decisions with management [43].

In Scrum, a self-managing team develops software in increments (called sprints); each sprint starts with planning, followed by performing tasks, and ends with a review. The team coordinates and makes decisions on a daily basis. Features to be implemented are registered in a product backlog, and a Product owner decides which backlog items should be developed in the following sprint. The product backlog defines everything that is needed in the final product based on current knowledge. It comprises a prioritized and constantly updated list of business and technical requirements for the system being built or enhanced. Backlog items can include features, functions, bug fixes, requested enhancements, and technology updates. Multiple stakeholders, such as clients, project teams, marketing and sales, management, and support [1], can participate in the planning phase to identify the product backlog items. Prioritizing the backlog is a complex communication and negotiation process; however, the Product owner is responsible for the final prioritization. During the planning meeting (usually every second or fourth week) the Product owner is responsible for presenting a prioritized product backlog. The highest priority items from the product backlog are then detailed in a sprint backlog by the developers.

While the product backlog is constantly updated, the sprint backlog should not be changed during the sprint, unless critical business requirements suddenly change, or if the team is not able to deliver as planned. The Scrum team members are empowered and expected to make day-to-day decisions within the project. They are also expected to always select the task with the highest priority when commencing work on items in the sprint backlog.

3.2.1. The South Company

The South Company is a medium-sized software company with approximately 150 employees in four organizational units. South was a plan-driven company, which used a process model based on ISO standards and fundamental concepts of the Capability Maturity Model (CMM). This process model is described in detail in [33]. A developer described how projects were run:

I wrote lots of plans using an enormous number of checklists. I could write plans for six months before the developers could even start writing code, and then we did not look at them anymore. All the stakeholders were involved in writing the plans. When I needed to change them, I had to ask every stakeholder about the change, but they were not interested in looking at a detailed plan of ten pages. It was terrible to update them. Creating and maintaining them took more time than coding. It was a waste.

In the South Company each developer was fully allocated to one project, and when there was a need for additional competence, for example on user interface or modeling, additional internal resources were used.

The goal of project A was to develop a system for integrity management of pipelines, both off-shore and on-shore. Today several clients are interested in buying the software product, and at the time of the study three contracts had been signed. The project involved developing a new system while at the same time trying to obtain input from the marketing department and potential clients.

The aim of project B was to develop a system for designing and maintaining off-shore installations. The project team had a very good relationship with their current client, who let the team make several of the decisions like reprioritizing releases without over-involving themselves. One of the developers worked as a Scrum master, and the former project manager became the Product owner. The Product owner was also responsible for system and acceptance testing, and for handling traditional project management tasks. The team used a 'Scrum wall' for planning and for keeping track of progress (Fig. 1).

3.2.2. North Company

In the North Company, project D started after project C had finished, with some of the developers participating in both projects. Project C was their first experience with Scrum. Before Scrum the projects followed a development model inspired by the waterfall model [41], and developers traditionally worked alone or on smaller projects. As in the South Company, the developers were highly specialized with corresponding divisions of work. In addition each developer usually worked on at least two projects. The developers spent from 50% to 80% of their time on the projects under study.

The goal of project C was to develop a plan and a coordination system, with a combination of textual user interfaces and map functionality. The client was the administration department of a major Norwegian city, and the North Company was also to become responsible for maintenance and support after final installation. A more detailed description of project C can be found in [32].

The goal of project D was to redevelop an information system to handle reports from clients. Winter was the low season for the use of this software; the high season began in spring (March/April) with 60–70 simultaneous users. The seasonal constraints gave a relatively narrow time frame for the introduction of a new version of the software. The project team initially consisted of two developers (one recently hired junior developer and one experienced developer). Two months after start-up, a third experienced programmer joined the team working from 50% to 100% of time on the project. Two of the developers, the Scrum master and the Product owner, also participated in project C.



Fig. 1. The Scrum wall of project B showing the task status and task flow.

3.3. Data collection

In order to understand the challenges of shared decision-making in agile software development, the first author conducted 45 semi-structured interviews with developers, Scrum masters and Product owners, each interview lasting from 20 to 50 min; in total 24 h. Everyone involved in the project was interviewed except for the remote developers in the Indian team in project A, and one of the consultants in project B who was absent at the time of the interview. All interviews were recorded and fully transcribed. The interview guide was based on a teamwork model proposed by Dickinson and McIntyre [9], which includes the most common elements that are considered in most research on teamwork processes and can be used on self-managing teams [32]. The interview guide also covered questions related to Scrum (Appendix A).

The first author also observed daily meetings, planning meetings, and review meetings. Participant observation was selected as a research technique to investigate the phenomenon of interest. It is a particularly relevant approach when ‘the phenomenon is obscured from the view of outsiders’ [16]. During coffee and lunch breaks the first author also discussed status, progress, and how issues were perceived by team participants. Telephone and e-mail were also used for such discussions. Notes were taken during dialogs, interactions, and activities. A short summary was written after each observation. Also most of the meetings were recorded and transcribed to make it possible to conduct analyses of the decision-making taking place in these meetings.

All notes, transcripts from interviews and meetings, were then integrated to produce a detailed record of how the decision-making was performed. The first author also collected Scrum artifacts, such as product backlogs, sprint backlogs, and burn-down charts. The data was collected over a period of 11–12 months in all projects. An overview of observations and interviews can be found in Table 2.

Table 2
Interviews and observations.

	South Company		North Company	
	Project A	Project B	Project C	Project D
Interviews	11	10	12	12
Observations	10	10	75	45

3.4. Data analysis

All the transcribed data from the interviews, observations, and documents was imported into a software tool for analyzing qualitative data, NVivo (www.qsrinternational.com). The first author read all interviews, observations and documents and coded interesting expressions of opinions, problems, events, happenings, or actions/interactions in the text by assigning the piece of text to a category (“node” in NVivo). A category represents a phenomenon, that is, a problem, an issue or an event that is defined as being significant. When categories were found to be conceptually similar in nature they were grouped under more abstract, higher-order categories. Finally, the first author used NVivo to create connections between categories and their subcategories.

The first author used a variety of strategies to analyze the material [21]. One such strategy was to describe the project and its context in a narrative to achieve an understanding of what was going on in the project. In the analysis, the first author emphasized how events were interpreted by different participants in the project. In this study, the interviews and observations were compared to each other. By doing this, some phenomena started to emerge, which all the authors then jointly compared to existing theories on decision-making. We then identified, analyzed, and reported patterns (themes) that emerged within the data by using thematic analysis [8].

By analyzing these patterns we identified challenges of shared decision-making in agile software development projects. An example of the coding process can be found in Fig. 2.

4. Decision-making in agile projects

We categorized the observed phenomena of product and project decision-making into the following three decision rich processes (categories):

- The process of aligning strategic plans with iteration plans;
- the process of allocating resources; and
- the process of performing tasks.

4.1. Aligning strategic and iteration plans

The participants in the alignment of the strategic and iteration plans are the Product owner and the team, and the process is enabled through continuous planning and feedback.

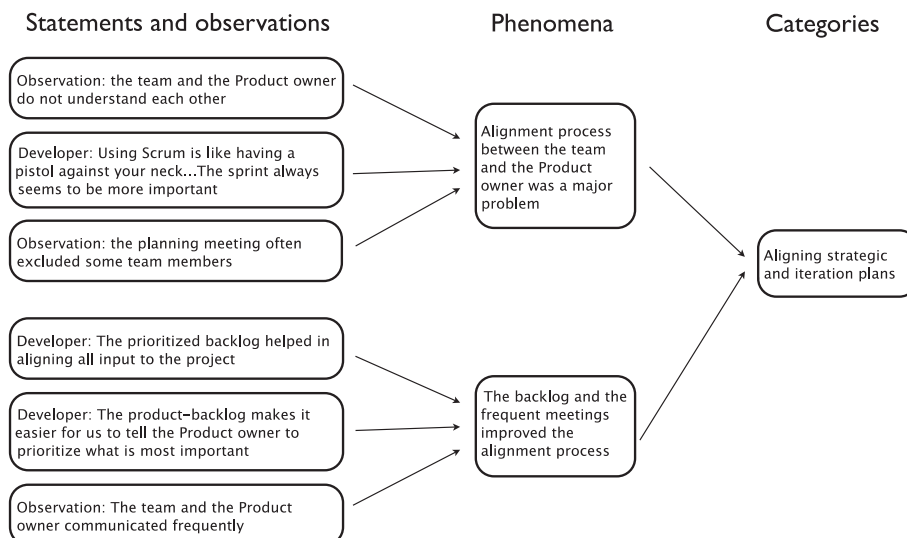


Fig. 2. Example of the coding process.

This alignment process between the team and the Product owner was a major problem in project A. The Product owner was from the business development department, and was responsible for the product strategy and talking to potential clients. The team felt that the Product owner did not clearly determine what was most important, and they believed that the presented features were too big and too abstract. Speaking to the Product owner, we found that he felt that the team lacked competence and therefore did not understand what he tried to communicate. Because the team had problems transforming the features presented about integrity management of pipelines into understandable work, the planning meetings often became unproductive and the team members became frustrated, because they could not make realistic plans for the upcoming iteration. Also, the team had to communicate frequently with the Product owner during the sprint to clarify the uncertainties. Some iterations used up a lot of time trying to understand what was going to be developed. There was little progress.

A few months after Scrum was introduced, the Scrum master in project A improved the situation by arranging a pre-planning meeting with the Product owner. The goal was to make sure that the prioritized list presented by the Product owner contained all work that the team was supposed to do in the sprint, and that they could understand the items on the list and translate these items into pieces of work. The Scrum master said:

Everything cannot be equally important. The product-backlog makes it easier for us to tell the Product owner to prioritize what is most important. . . The prioritized backlog helped in aligning all input to the project.

While the process improved, the team and the Product owner still discovered several misunderstandings during the iteration. According to Scrum, if goals and strategies are changed in the middle of a sprint, the sprint should be terminated, and a new one should be initiated. However, none of the sprints in project A were terminated. Also, the team felt that the Product owner frequently changed his priorities (strategic plans) after talking to new potential clients when trying to sign new contracts, and these potential clients often had conflicting needs. Accordingly, the team became even more frustrated. The team felt that the Product owner had problems with processing and prioritizing feedback from various clients. While the situation was challenging, the team felt that the backlog helped them to protect the iteration from changes. A developer said:

The sprint and sprint backlog makes it easier to say “no” to the Product owner.

The situation in project B was much more predictable, partly because the system for designing and maintaining off-shore installations had only one client, and the Product owner was good at describing and prioritizing the client's needs before the team started on a new release. As a consequence, each iteration had a clear goal, and the team knew what they were supposed to deliver in the next iteration. However, it seemed difficult to prioritize activities that would improve long-term quality, when these activities would reduce the pace of developing new features. One developer in project B commented:

Using Scrum is like having a pistol against your neck. It's good and bad. You fix things now and not later. But there are also tasks you should have done like code refactoring. I think we do not use enough time on refactoring, because you need to deliver what you promised. We always promise a little too much. During our meetings it is difficult to argue for investing resources in doing such tasks. The sprint always seems to be more important.

While the goal of a particular sprint is to give the most business value to the client (immediate value creation), sometimes this seemed to be in conflict with the strategic goal of giving the client the most value in a longer term.

This conflict between the need for progress and the need for long-term quality could be why projects A, C, and D experienced problems with software quality and the code being disorganized. These problems also affected the alignment process. Adding new features meant changing the existing code, which often resulted in a lot of unexpected work, e.g. discovery of new bugs. So implementing a feature could end up taking one or two weeks instead of one or two hours as planned.

To understand better the client needs and goals in project D (redevelopment of an information system), the team arranged extensive design workshops and used white-board prototyping to create sketches of the user interface. This made it easy for the Product owner and the team to discuss project goals and product strategy, and to create a shared understanding of the work to be done. However, the team still experienced problems, especially in relation to the planning process. The Product owner was situated in another city, and he did not prepare well enough for the planning meetings. In these meetings, the first author observed several discussions about the purpose of each feature and how the system was supposed to operate. It was important to decide on what to include in the iteration, but these discussions were time-consuming and seemed somehow to exclude the developers. The first author even observed a developer falling asleep. Often the Scrum master and Product owner in project D ended up discussing issues that should have been clarified before the meeting. As a consequence there was often too little time available for planning the upcoming sprint, and it probably affected the team's chance of really committing to the sprint. Also the Product owner had strong opinions on how the developers should perform their tasks, and therefore described features at a very low level. One developer in project D commented:

The product backlog is a mixture of very detailed and very general items. Especially all the detailed items are a problem because they also include design. It should be us splitting up the features not the Product owner.

The effect was that developers often ended up working on several of the small items in parallel, because these items were frequently strongly connected. When most of the work was “in progress” the team lost the big picture view of the overall sprint progress.

Project C suffered from severe problems with integration of a component (map functionality) delivered by a sub-contractor. One reason was that management did not understand the complexity of the system being developed at the time of signing the contracts with the client and the sub-contractor. When planning the project, it was agreed that the sub-contractor would deliver the most critical component as late as one month before first installation. Because the sub-contractor was also a competitor, management decided not to give them details about project plans and goals. The effect of this strategy was almost no communication between the developers and the sub-contractor, and significant integration problems at the end of the project.

4.2. Allocating resources

The teams decided themselves how to organize the resources given to them by management. Short iterations helped the teams to protect their resources. The Product owner of project B explained:

It is now more difficult to “steal” resources from us, because the consequences of losing resources during a sprint are more visible with Scrum. Earlier the deadline was 6–12 months ahead, and then it was easy to steal a day or two. There has been a change of attitude in the company, and it is now well accepted that you do not steal resources from a Scrum team during a sprint.

When discussing who should do what, the Product owner in project A argued that people should be specialized with a corresponding division of work. There should be different people responsible for the databases, a user interface expert etc. In his view this was the most effective way of developing software. The team did not agree. The Scrum master in project A focused on the need for redundancy. He said:

Often it is obvious who should do what. However, we need to think of the consequences of letting the database person only work on the database. We then get too vulnerable, and it will reduce his job satisfaction. We need to let people work on the things that motivate them. This is something we have changed after introducing Scrum.

However it was a challenge to change the company culture. The Scrum master continued:

I perceive the team as not taking real responsibility. When they have finished a task they ask me “what should I do now?”. I would expect them to take more responsibility. They are used to be given single tasks. This is how we used to work. They do not see all the tasks as a pool they can choose from.

Also in project B although the team was required to select the most important task, the tasks were usually assigned to developers during the planning meeting and not chosen during the sprint. One developer from project B said:

We try to identify who is most qualified to do what during a sprint. During the planning meeting I say: “I think I should work on this”. And when we are correcting errors I do that, since I’m the most experienced.

This view on specialization started to change, but the need for progress and the cost of training was still a strong argument for keeping people specialized. The Product owner in project B said:

If someone has finished all their tasks, then they can help the others. However they should not pick new tasks if it requires too much training. If so, we identify other tasks suggested in the next sprint and we postpone the things that are not critical for this sprint. Sometimes it is important to invest in training since it makes people more comprehensive.

In project B they hired two external consultants because they lacked resources internally. Both in the interviews and in the observations the first author found that these consultants were treated differently than the rest of the team. The company did not want to invest in building competency among the consultants, and subsequently the consultants were not allowed to select freely from the pool of tasks, i.e. the consultants were most commonly assigned tasks.

In project D, when starting a new sprint the team identified several conflicting dependencies (e.g. on technology, sub-contractors, and other projects) caused by the complex environment of the project. Consequently, the Scrum master in project D suggested adding more tasks to the backlog than the team could possibly handle during the sprint. He said:

...there were so many dependencies on other projects and sub-contractors, so we did not know which tasks we could complete during the sprint. It is also about optimization. You should always have the possibility to work on something else if you are stuck. Therefore we wanted to decide as late as possible what to work on.

His goal of optimization resulted in the team losing sight of the big picture view of the sprint and the sprint goal, and subsequently the team was unable to identify the most important tasks to work on. One developer in project D commented:

Since we also added so many features we knew we could not finish, we did not care if we did not complete all the tasks during a sprint. Some tasks were moved four sprints before they were even started.

Also the team was not strict about the definition of ‘done’, i.e. when a task was finished. One developer in project D commented:

We classified tasks as finished before they were completed, and we knew there was still work to be done. It seems that the Scrum master wants to show progress, and make us look a little better than we really are. These tasks are then not on the list of the next sprint since they officially are done. Each sprint starts with doing things that we have said were finished, and then you know you will not finish the sprint.

In projects C and D the team had problems losing resources during the sprints. Some developers worked on additional projects, and some developers needed to provide support for products they had worked on earlier. When crisis emerged elsewhere in the organization, projects C and D often ended up with fewer resources than initially planned. Solving problems fast was seen as a competitive advantage by management, however it affected the alignment of decisions on the project and product level. Often the developers needed to decide themselves which project they should give priority. Both the team and the Product owner in projects C and D were frustrated with this situation.

4.3. Performing tasks

The daily meeting was seen as the best means to improve team communication, to spread knowledge, and to align decisions in the team, since the objective of the meeting was to clarify problems, resolve uncertainties, and to engage in decision-making. If the team could not solve a problem, it was discussed with the Product owner. The Product owner decided when a bigger problem should be solved and where to allocate resources. The frequent dialogs also resulted in early identification of problems at the operational and tactical levels. The Product owner of project A said:

If the team gets stuck, and does not know what to do because of missing or conflicting decisions, this is discovered early... Before Scrum, developers could use months before we discovered that there were problems we needed to discuss.

Also in project B, there was a strong focus on asking for help and addressing problems early. When a problem was reported at a daily meeting, everybody participated in the discussion. If the team realized that the problem was only related to a few of the team members, the persons involved usually continued the discussion after the meeting. In project B problems were not just reported during the daily meetings. The Product owner said:

It’s OK to use ten minutes solving a problem, but if you then still are stuck, you should get up and ask for help. And usually someone knows. You do not lose face by asking, this is what I nag about... If the team can decide themselves, I’m not involved.

While problems were reported frequently, the first author found that not all problems were reported or discussed. When discussing solutions to a problem related to a feature or a task implementation, the first author observed experts expressing their opinion without the issue being debated by other interested parties. In some cases, team members made decisions without informing others or without being challenged by others. The Scrum master in project A talked about a developer making decisions without informing the team:

We divide tasks among ourselves, and then people are responsible for implementing them. This usually works fine. However, in the last sprint one of the developers spoke with someone in the market about an issue. The developer used a couple of days changing a feature that was already finished, because the developer thought this was very important. While testing I found out what had happened and we had to redo what the developer had done.

In Project C the Scrum master eventually became aware of problems not being reported. He tried to improve the development process by paying more attention to the nature of the problem and by discussing this with the team. However the developers felt the Scrum master was overreacting and creating a lot of noise, so they began reporting even fewer problems. This in turn affected the decision process in project C since problems were not discussed at the daily meetings. Even when problems were identified, the first author observed that some of them were not ultimately solved. One of the developers in project C said:

When we discuss, usually we identify all the important issues. However no one seems to take the responsibility for making sure these issues were handled. Not even the Scrum master seemed to be aware of this. Everyone was focusing on their own work.

Another reason for problems not being reported or discussed was that in many cases module specific problems were seen as personal problems. Also some developers did not want to argue with the others over solutions. This was especially evident in project C. One developer commented:

When we start discussing how we should solve programming problems, sometimes we get stuck. It's like religion, and then you end up letting people do what they want, because you do not have the energy to do the fighting.

Lack of knowledge also seemed to cause problems in this shared decision-making environment. One developer in project B explained:

When we use planning-poker we do it too fast and without preparation, we only suggest numbers without really knowing. The tasks are more complex than we realize. Because we lack knowledge about the problem to solve, the meetings are time consuming. And when I propose an estimate I do not get really valuable feedback from the others.

When team members lacked knowledge about a task, they were unable to participate in the decision-making process. The main reason for this lack of knowledge was that traditionally the companies relied on specialization. A developer usually worked on his own module and was seldom involved in the work of others. Another reason for developers lacking knowledge was that the company did not have a strategy for training new team members.

Focusing on their own work resulted in developers having problems giving feedback, and in some of the developers not paying attention to what others had to say during the daily Scrum meetings. One developer said:

When it comes to the daily Scrum, I do not pay attention when Ann is talking. For me it is a bit far off what she talks about, and I do not manage to pay attention. She talks about the things she is working on. I guess this situation is not good for the project.

5. Discussion

We have described decision-making in four agile projects with the focus on understanding shared decision-making and how decisions are aligned on the strategic, tactical and operational level within the project. Based on our findings, we identified three main challenges to shared decision-making in agile software development that will be discussed in this section: alignment of strategic

product plans with iteration plans, allocation of development resources, and performing development and maintenance tasks in teams. Together, these challenges suggest that moving from plan-driven to agile development can be seen as moving from rational to naturalistic decision-making.

5.1. Challenges in aligning strategic product plans with iteration plans

Aligning strategic, tactical, and operational decisions depended on timely feedback between business and technical level decision makers. This was supported by frequent planning meetings and frequent feedback between the teams and the Product owners. The importance of feedback for aligning decisions on all levels is confirmed by the findings of Aurum and Wohlin [3]. While the agile process supports the alignment between the strategic and the operational levels, it was nevertheless found to be difficult in practice. The main reasons for problems aligning strategic and product plans were:

- Lack of shared understanding.
- Not understanding the complexity.
- No arena for solving conflicts.
- Not involving the team.

5.1.1. Lack of shared understanding

The main reason for the problems with the alignment between the strategic and the operational levels was the lack of a shared understanding of what was supposed to be delivered in the upcoming release. Because the Product owner and the team are jointly responsible for the end product, they need to develop shared mental models by negotiating shared understandings about both the teamwork and the task [23]. However, the projects underestimated the importance of creating such shared mental models before the iteration plans were settled, and therefore projects ended up with unrealistic plans, plans that changed too frequently, and team members not committing to the plans. Our finding confirms the results of previous research on software development teams, i.e. that not all teams develop shared mental models over time [23]. We found three possible explanations for the lack of shared mental models.

5.1.2. Not understanding the complexity

One explanation for why shared mental model were lacking, could be that both the team and the Product owner did not realize the complexity of the requirements being specified for each iteration/release. The planning meeting was not sufficient for making everybody involved understand the complexity, and this resulted in changes in the plans during an iteration. When unpredictable situations emerge or when plans are found to be faulty, teams should have the ability to change strategies [28]. However, if this happens during an iteration, the iteration should be stopped and planned again. This never happened in the projects that we studied.

Some of the requirement complexity can be attributed to the technology that has grown in both scale and scope in the last decade. Also, in the projects under study, the team and the Product owner needed to consider the viewpoints of a wide variety of stakeholders, many of whom had conflicting views on the desirability of the software features and its functionality. This complexity is the reason why today software development is often referred to as solving “wicked problems” [35]. Although shared mental models may develop over time, they can also be instantiated effectively through training [27]. In the North Company they used initial workshops to strengthen the shared mental models in order to understand and to agree upon what project teams were actually going to deliver.

5.1.3. No arena for solving conflicts

A second explanation for lacking shared mental models, was no arena for solving conflicts between decisions on strategic, tactical, and operational project levels. One example was the frequent conflict between the need for long-term quality (strategic level) and short-term progress (tactical level). This conflict is commonly found in software projects because of the dynamic relationships between software processes and the three outcome factors: cost, schedule, and quality [19]. Change-driven development does not improve the quality of the software or the productivity of the developers (Li [24,32]), however in agile development this conflict appeared at the end of every iteration. It was found that the need for long-term quality was given a lower priority than the need for short-term progress. This is in agreement with Ramesh et al. [40], who found that agile practices result in non-functional requirements being neglected. Also, the iteration pressure caused developers to stop following the process implemented (i.e. not proper testing). This is in agreement with Zazworka et al. [50] that received the following answer when asking developers why they did not follow agile practices: “the implementation of new features to satisfy customer needs had a higher priority than following the steps of the process” p. 8.

There seemed to be three main reasons for why it was difficult to align operational, tactical and strategic decision related to quality and amount of new features. First, the team felt that their primary goal was to deliver according to the release plan and that they always needed to demonstrate that they were making ‘progress’. This could also be one explanation as to why Scrum masters and team members sometimes tried to give the impression that the team was doing better than it actually was. Second, discussing and changing the release plan was seen as troublesome, because it meant involving top-level management, steering group, and clients. Third, it was observed that the product owner seldom expressed quality as an explicit requirement. Apparently, it was considered so obvious that often it was not even mentioned.

5.1.4. Not involving the team

A third reason for problems creating shared understanding, was that the organizations were not used to involving the teams when making strategic project decisions. In the North Company, we found that team C was not involved when management made an important decision about a significant third party component. Management did not understand the consequence of their decisions, and the project ended up with several serious problems in the last phase. All teams were accustomed to plan-driven development, but were usually not involved in strategic decisions (Fig. 3). In plan-driven development decision-making is governed by the

hierarchical command and control structure, while in change-driven development team empowerment and shared decision-making is encouraged at all levels. Introducing agile software development and self-management means changing organizational culture, strategy, and structure. Such changes in a development organization take several years [47], which might explain why the organizations were partly adhering to their old ways of decision-making.

5.2. Challenges of allocating resources

While the team in project B reported that agile software development helped them protect their resources, projects A, C and D ended up with unrealistic sprint backlogs that contained too many tasks and too few resources. The unrealistic sprint backlogs resulted in problems allocating resources, because no one knew when a team could deliver, or how to prioritize between the project and the rest of the company. The main reasons for unrealistic backlogs were:

- Important decisions not aligned.
- Missing a clear prioritization.
- Missing a definition of done.
- Conflicting priorities within the company.
- Low committing to the plan.

5.2.1. Important decisions not aligned

First, the Scrum master in project D postponed the alignment of important decisions (e.g. related to the sub-contractor) and therefore added too many features to the sprint backlog. Paradoxically, this was done because he wanted to achieve adaptability through a more flexible sprint backlog. According to this view, important strategic decisions should be delayed as much as possible in order to allow a more flexible response to last minute feedback from the market place [45].

5.2.2. Missing a clear prioritization

Second, when conducting the planning meetings the Product owner seldom provided the team with a well-prioritized product backlog aligning all decisions. As a consequence, the planning meetings ended up using a lot of time on discussing the scope of the project, identification of features, client needs, and technical problems. Although this was important for the Product owner to discuss, the objective of these meetings was to implement the strategic decisions, i.e. tactical decision related to how to perform tasks and how to allocate resources. As a result there was too little time for planning the upcoming sprint.

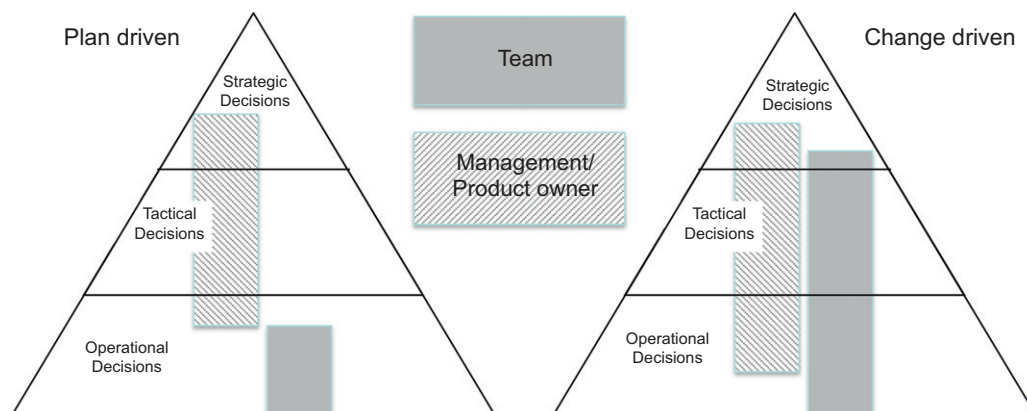


Fig. 3. Plan- and change-driven development in Anthony's [2] model of decision-making.

5.2.3. Missing a definition of done

Third, projects C and D were missing a clear definition of 'done' and the Scrum master often wanted to make the team look better than they actually were. Therefore, each iteration started by performing tasks that were officially finished in the previous iteration, and then everybody in the team knew they could not complete what was planned for the current iteration.

5.2.4. Conflicting priorities within the company

Fourth, projects C and D lost resources during iterations because of conflicting priorities within the company. The teams were lacking support from the organization and adequate resources, which are two important factors for achieving self-management [14]. Not aligning tactical decisions on the company level resulted in challenges on the project and product level.

5.2.5. Low committing to the plan

As a consequence of the unrealistic sprint backlog, the developers did not commit to the plans and started to perceive their own individual plans as more important than the team's plan, which is a clear sign of lack of team orientation [42]. The unrealistic plans also resulted in the teams not having a clear direction, and a clear direction is one of the prerequisites for being able to self-manage [14]. When a team was lacking team commitment and when the team tried to look better than they actually were, this made the plans even more unrealistic. At the end of a project, team-members joined new projects before the current one was done because they were already allocated on new assignments several months before.

5.3. Challenges of performing tasks together

The change-driven approach was found to provide a strong infrastructure for continuous problem solving and decision-making, since it encouraged frequent reporting of problems, and the awareness of a problem is often the first important step towards a solution. The goal of frequent reflection is to make the team continuously correct failures and potentially modify the development process based on experience [6]. One example is project B, where developers were required to seek help if they were unable to make progress for more than ten minutes. While problems were frequently reported, it was still a challenge to implement effective and shared decision-making at the operational level. The main reasons were:

- Not confronting each other.
- Lacking team orientation.
- Unrealistic plans.
- Decision-hijacking.
- Lack of knowledge.
- Technocracy.

5.3.1. Not confronting each other

One reason for problems with the decision-making process was that some developers did not want to argue with the others, and some developers found it more important to conform with the other group members. As a result the teams experienced ineffective decision-making. This consistent with the findings by McAvoy and Butler [29] on reasons for ineffective decision-making.

5.3.2. Lacking team orientation

Another reason for problems with the decision-making process was that some teams had issues with their team orientation and missed a clear direction, which is important when the team performs tasks together. Team orientation was generally strong because the daily meetings encouraged the team and team members to ask questions about what they were doing. However,

at times team orientation was low for some teams because of unrealistic plans, and in one team people even fell asleep at daily meetings.

5.3.3. Decision-hijacking

Low team orientation could be one reason why in projects A and C the teams experienced decision-hijacking [4], i.e. some team members making decisions without informing the others. Low team orientation was probably also the reason why problems were seen as personal and not reported, which hindered participation in decision-making.

5.3.4. Lack of knowledge

Lack of knowledge is another reason why people were prevented from participating in the decision-making. When developers lacked knowledge they could not give feedback to others when discussing problems or give advice on how to perform a task. Consequently, they did not pay attention when problems were reported, and subsequently could not participate in the shared decision-making process. The main reason for the lack of knowledge about the work of others was specialization and the corresponding division of work.

5.3.5. Technocracy

Shared decision-making was also hindered because the organizations were often challenged by technocracy [34], where expertise is the basis of authority. The high expert power attained by certain skilled software practitioners was therefore a major road-block for aligning decisions and for the establishment of shared decision-making and self-management. The high expert power also seemed to enable super-decisions [29]. These are decisions which are based on assumptions that are unchallenged while maintaining consensus.

5.4. Implications for research and practice

This multiple case study has a number of implications for research and practice. For research this study shows a clear need for more empirical studies of naturalistic decision-making in agile development. There are two reasons for this. First, today's software systems are becoming more complex because of the need to balance the often disparate needs of diverse stakeholders, and the growing complexity of the technology used [35]. This reality requires more unstructured decision-making. Second, the principles of agile software development align with the definition of naturalistic decision-making [49].

The agile software development approach emphasizes that teams should be self-managed and committed to the iteration or sprint plan. However, Scrum and agile methods offer no advice on how to implement this, and especially no advice on how to implement a shared decision-making process. We found that specialization and lack of training hinder a shared decision-making process and the potential of committing to plans. Commitment was also problematic when the iteration plans are perceived as unrealistic and the team is missing a clear direction. In addition Scrum does not solve the challenge of combining long-term and short-term goals, which was seen as a problem in the projects. There is a need for more empirical studies in these areas.

Our findings also show that introducing Scrum might require changes at all levels of the organization: from how resources are allocated, how the organization supports the teams, and how backlogs are aligned with strategic decisions, to power shifts in decision-making, and to the implementation of a shared decision-making process at the operational level. It also takes time to change the decision-making process. The organizations that were the subject of our case studies needed from 1 to 2 years

before they were able to change how they had traditionally conducted decision-making in software development. Even at the end of our study, there were still issues to be resolved in order to run Scrum more smoothly.

Our findings showed that Scrum provides an environment for bringing stakeholders from all management levels together, which may lead to a better alignment of strategic, tactical, and operational decisions. However, Scrum focuses only on agile project management and the software development team. In order to implement Scrum successfully, it is imperative for the agile approach to be adopted effectively at the organizational level as well. First, misalignment between the team structure and the organizational structure can be counterproductive, e.g. when team goals are in conflict with organizational goals, it may threaten the autonomy of the development teams [31]. Second, the specialized cultures made it difficult to change how teams operated. Because building redundancy requires extra resources, this should be the responsibility of the organization [13]. Third, the Product owner and the team need to allocate enough resources to be able to build a common understanding of what will be delivered in the upcoming iteration/sprint, and to make sure that strategic decisions are aligned with the tactical and operational decisions when commencing the planning meeting. A one or two day planning meeting is often not enough.

We believe that our study also has important implications for practice. First, the planning meetings need to be well planned by the Product owner and the team to enable everybody to participate in decision-making on the tactical and operational levels. Both short-term and long-term goals are important and must be aligned. Second, there is a need to reduce the amount of work in progress. When the code base is unstable or when it is difficult to understand the work required to implement a feature, iterations can be shortened or the number of backlog items committed to can be reduced. Third, all team members should be able to participate in the decision-making process. This is essential for the team to self-manage and to be able to align decisions on all levels. Also, because developers play a much more important role in practice than what is recommended in the literature when it comes to decision-making [39], they need to be included in the whole decision-making process. One of the key aspects of XP, for example, is the clear separation of roles between the business and technical people. Fourth, it is important to create a company culture that builds redundancy, and that reflects on how to solve process related problems at daily meetings. It is not enough to report only problems related to technical issues.

5.5. Limitations

The general criticisms about case studies, such as uniqueness and special access to key informants, may also apply to our study. According to Lee and Baskerville [22] generalizing can occur in four ways: From empirical statements to other empirical statements, from empirical statements to theoretical statements, from theoretical statements to empirical statements, and from theoretical statements to other theoretical statements. In this study we relied on the first type, which involves generalizing data to an observation. One example is how we investigated shared decision-making by comparing what people said in interviews and lunch breaks, with what we observed.

The rationale for choosing two companies and four projects in these companies was that they represented critical cases for exploring the challenges of shared decision-making in agile software development teams. All teams developed products based on different technology and customer relationship. Project A developed a new product for a new market, project B continued developing a product for the same customer, product C was a tailored

project for one specific customer and project D was a redevelopment of an already existing product with an internal customer. Because the same phenomena was reported across all teams, it is likely that other medium-sized companies will find the phenomena reported on, and the conclusions in this study useful.

Another possible limitation is that much of the data collection and analysis was based on semi-structured interviews and participant observation. The consequence of this limitation is that the results are under the influence of our interpretation of the phenomena observed and investigated. The use of multiple data sources made it possible to confirm evidence for episodes and phenomena. The study included observing, talking to, and interviewing team members and managers in all companies and all projects, which made it possible to investigate the phenomena from different viewpoints as they emerged and changed, thus reducing this limitation. There is a risk that our findings also can be explained by factors that evaded our attention. One reason is that we did not collect data from all meetings or all activities in the projects, and therefore probably missed some events. However, giving feedback to the observed teams and discussing our interpretation of what was going on helped with validating our conclusions.

It is possible that the first author, as a researcher, influenced the teamwork characteristics by his presence in the project. Although his presence and questions might have made the team members more aware of the decision-making process, we do not think their behavior was influenced by the first author's presence. The main reason for this is that the companies were studied over a long time period (11–12 months in all four projects), and that the first author already had a long standing relationship with both companies. In other words, his presence was not unusual for the participants and is less likely to have triggered such a self-conscious response in the participants.

5.6. Future work

This study highlights challenges of shared decision-making in agile software development. Aligning decisions on all levels and implementing a shared decision-making process on an operational level is essential for the success of agile development. Accordingly, further work should focus on investigating other decision-making challenges associated with introducing agile development. There is a particular need for better understanding why agile teams have problems aligning decisions on all levels, handling the iteration pressure and why the team-members and management have a different view on the balance of quality and new features. To be able to answer such questions, there is a need for more research on the decision-making process in more mature agile teams and organizations, e.g. understanding the decision-making process in a fully self-managed team.

Company A is a global company currently applying agile methods to their distributed projects to meet challenges related to decision-making, communication, and coordination. This topic is not well covered in the agile literature. An area for future research is therefore to understand the decision-making processes when applying an agile methodology in global software development.

6. Conclusion

This paper presented data from a multiple case study and the goal was to explore and form a base for understanding the challenges of decision-making in agile software development.

The evidence shows that the decision-making processes differed in each project, although the outcomes were similar. For example, there was often a conflict between the need for

short-term progress and the need for long-term product quality at the end of sprints in three of the four projects, which in turn made it difficult to align decisions on the operational level, and between the operational, tactical, and strategic levels.

We also found that self-management was affected by the ability to implement a shared decision-making process. When the teams were missing a clear direction (e.g. unrealistic plans and plans without a clear priority), individual goals often became more important than team goals, and alignment among all levels seemed to fail. Introducing shared leadership and shared decision-making does not mean that everyone needs to be involved in all decisions, however all important decisions must be communicated to the whole team, and the team needs to identify which decisions need to be taken together. We found that several team-members were left out of important decisions. Also, agile development is designed for managing project development, not for resolving company internal or cultural problems, e.g. expertise as the basis of authority (technocracy) and problems related to losing resources.

Changing the way of working is difficult, and when it involves a transition from specialized skills to redundancy of functions and from rational to naturalistic decision-making, it requires a reorientation not only by the developers but also by management. This change takes time and resources, and it must be implemented to be able to succeed with agile software development. While introducing the agile approach to a software project is a top-level strategic management decision, it is also important that this approach is accepted and supported by the whole organization and all stakeholders at the management and the operational levels.

Acknowledgment

We appreciate the input received from the project participants of the investigated companies. This research is supported by the TeamIT project and the Research Council of Norway under Grant 193236/140. We are grateful to Torgeir Dingsøy from SINTEF ICT, who participated in the data collection at the North Company, and to the reviewers of this paper who gave us valuable feedback.

Appendix A. Interview guide

The respondent was informed of the nature of the study and how long the interview will take. The respondent was told why it is important to tape the interview and that only the researchers would have access to the transcript. The respondent was finally asked if he/she would agree to the interview being taped.

Questions for warm-up:

- What are you working on now?
- What is the status of the project?

The main body of the interview:

- How is work coordinated in the project?
- How are problems that emerge in the project solved?
- Do you have an overview of what others are doing?
- How easy is it to continue on others work?
- How do you discover changes in the project?
- How do you deal with changes in the project?
- Does the team have a common project goal?
- Does everyone know the expected outcome of the project?
- Do team members give each other feedback in the project?
- Do team members share relevant project information with each other?
- How is the team communication?
- How is the team performance?

- How do you think Scrum is working in the project?
 - What is working?
 - What is not working?
- Is there anything else you would like to add that you think is interesting in this context, but not covered by the questions asked?

In the first interview round the following question was asked after all questions in the guide: “How was it done in earlier projects?”

References

- [1] P. Abrahamsson, O. Salo, J. Ronkainen, J. Warsta, *Agile Software Development Methods – Review and Analysis*, VTT Publications, 2002.
- [2] R.N. Anthony, *Planning and Control Systems: A Framework for Analysis*, Harvard University, Boston, USA, 1965.
- [3] A. Aurum, C. Wohlin, A value-based approach in requirements engineering: explaining some of the fundamental concepts. Requirements engineering: foundation for software quality, in: 13th International Working Conference, REFSQ 2007. Trondheim, Norway, June 11–12, 2007. (Proceedings of Lecture Notes in Computer Science 4542, vol. 4542. Springer; 2007. pp. 109–115).
- [4] A. Aurum, C. Wohlin, A. Porter, Aligning software project decisions: a case study, *International Journal of Software Engineering and Knowledge Engineering* 16 (6) (2006) 795–818.
- [5] K. Beck, C. Anders, *Extreme Programming Explained: Embrace Change*, Addison-Wesley, 2004.
- [6] A. Börjesson, L. Mathiassen, Successful process implementation, *IEEE Software* 21 (4) (2004) 36–44.
- [7] A. Cockburn, J. Highsmith, Agile software development: the people factor, *Computer* 34 (11) (2001) 131–133.
- [8] D.S. Cruzes, T. Dybå, Research synthesis in software engineering: a tertiary study, *Information and Software Technology* 53 (5) (2011) 440–455.
- [9] T.L. Dickinson, R.M. McIntyre, A conceptual framework of teamwork measurement, in: M.T. Brannick, E. Salas, C. Prince (Eds.), *Team Performance Assessment and Measurement: Theory, Methods, and Applications*, Psychology Press, NJ, 1997, pp. 19–43.
- [10] T. Dybå, Improvisation in Small Software Organizations, *IEEE Software* 17 (5) (2000) 82–87.
- [11] T. Dybå, T. Dingsøy, Empirical studies of agile software development: a systematic review, *Information and Software Technology* 50 (9–10) (2008) 833–859.
- [12] S.M. Fjellman, Natural and unnatural decision-making, *Ethos* 4 (1) (1976) 73–94.
- [13] T.E. Fægri, T. Dybå, T. Dingsøy, Introducing knowledge redundancy practice in software development: experiences with job rotation in support work, *Information and Software Technology* 52 (10) (2010) 1118–1132.
- [14] J.R. Hackman, The psychology of self-management in organizations, in: M.S. Pallack, R.O. Perloff (Eds.), *Psychology and Work: Productivity, Change, and employment*, American Psychological Association, Washington, DC, 1986.
- [15] B. Hewitt, D. Walz, Using shared leadership to foster knowledge sharing in information systems development projects, in: *Proceedings of the 38th Hawaii International Conference on System Sciences (HICSS)*, 2005.
- [16] D.L. Jorgensen, *Participant Observation: A Methodology for Human Studies*, Sage publications, Thousands Oak, California, 1989.
- [17] B.L. Kirkman, B. Rosen, Beyond self-management: antecedents and consequences of team empowerment, *Academy of Management Journal* 42 (1) (1999) 58–74.
- [18] G. Klein, Naturalistic decision making, *Human Factors: The Journal of the Human Factors and Ergonomics Society* 50 (3) (2008) 456–460.
- [19] H. Krasner, The payoff for software process improvement: what it is and how to get it, in: K.E. Emam, N.H. Madhavji (Eds.), *Elements of Software Process Assessment and Improvement*, IEEE Computer Society Press, Los Alamitos, California, 1999, pp. 151–176.
- [20] K. Langfield-Smith, Management control systems and strategy: a critical review, *Accounting Organizations and Society* 22 (2) (1997) 207–232.
- [21] A. Langley, Strategies for theorizing from process data, *Academy of Management* 24 (1999) 691–710.
- [22] A.S. Lee, R.L. Baskerville, Generalizing generalizability in information systems research, *Information systems research* 14 (3) (2003) 221–243.
- [23] L.L. Levesque, J.M. Wilson, D.R. Wholey, Cognitive divergence and shared mental models in software development project teams, *Journal of Organizational Behavior* 22 (2001) 135–144.
- [24] J. Li, N.B. Moe, T. Dybå, Transition from a plan-driven process to Scrum: a longitudinal case study on software quality, in: *Proceedings of the 2010 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, ACM, Bolzano-Bozen, Italy, 2010, pp. 1–10.
- [25] R. Lipshitz, G. Klein, J. Orasanu, E. Salas, Taking stock of naturalistic decision making, *Journal of Behavioral Decision Making* 14 (5) (2001) 331–352.

- [26] J.G. March, A preface to understanding how decisions happen in organizations, in: Z. Shapira (Ed.), *Organizational Decision Making*, Cambridge University Press, 1996.
- [27] M.A. Marks, S.J. Zaccaro, J.E. Mathieu, Performance implications of leader briefings and team-interaction training for team adaptation to novel environments, *Journal of Applied Psychology* 85 (2000) 971–986.
- [28] M.A. Marks, J.E. Mathieu, S.J. Zaccaro, A temporally based framework and taxonomy of team processes, *Academy of Management Review* 26 (3) (2001) 356–376.
- [29] J. McAvoy, T. Butler, The role of project management in ineffective decision making within agile software development projects, *European Journal of Information Systems* 18 (4) (2009) 372–383.
- [30] H. Mintzberg, *Mintzberg on Management: Inside Our Strange World of Organizations*, Free Press, New York, 1989.
- [31] N.B. Moe, T. Dingsøy, T. Dybå, Overcoming Barriers to Self-Management in Software Teams, *IEEE Software* 26 (6) (2009) 20–26.
- [32] N.B. Moe, T. Dingsøy, T. Dybå, A teamwork model for understanding an agile team: a case study of a Scrum project, *Information and Software Technology* 52 (5) (2010) 480–491.
- [33] N.B. Moe, T. Dybå, The use of an electronic process guide in a medium sized software development company, *Software Process Improvement and Practice* 11 (1) (2006) 21–34.
- [34] G. Morgan, *Images of Organizations*, Sage publications, Thousand Oaks, CA, 2006.
- [35] S. Nerur, V. Balijepally, Theoretical reflections on agile development methodologies – the traditional goal of optimization and control is making way for learning and innovation, *Communications of the ACM* 50 (3) (2007) 79–83.
- [36] S. Nerur, R. Mahapatra, G. Mangalaraj, Challenges of migrating to agile methodologies, *Communications of the ACM* 48 (5) (2005) 72–78.
- [37] L. Neumann-Alkier, Think globally, act locally – does it follow the rule in multinational corporations?, in: 5th European Conference on Information Systems, Cork Publishing, Ltd, Cork, 1997.
- [38] C.L. Pearce, The future of leadership: combining vertical and shared leadership to transform knowledge work, *Academy of Management Executive* 18 (1) (2004) 47–57.
- [39] Z. Racheva, M. Daneva, S. Klaas, W. Roel, A. Hermann, Do we know enough about requirements prioritization in agile projects: insights from a case study, in: 18th International IEEE Requirements Engineering Conference, IEEE Computer Society Press, Sydney, Australia, 2010, pp. 147–156.
- [40] B. Ramesh, L. Cao, R. Baskerville, Agile requirements engineering practices and challenges: an empirical study, *Information Systems Journal* 20 (5) (2010) 449–480.
- [41] W.W. Royce, Managing the development of large software systems, in: *Proceedings of IEEE WESCON*, vol. 26, August, 1970, pp. 1–9.
- [42] E. Salas, D.E. Sims, C.S. Burke, Is there a “big five” in teamwork?, *Small Group Research* 36 (5) (2005) 555–599.
- [43] K. Schwaber, *Beedle, Agile Software Development with Scrum*, Prentice Hall, Upper Saddle River, 2001.
- [44] H.A. Simon, A behavioral model of rational choice, *The Quarterly Journal of Economics* 69 (1) (1955) 100–118.
- [45] H. Takeuchi, I. Nonaka, The new product development game, *Harvard Business Review* (64) (1986) 137–146.
- [46] J. Tata, S. Prasad, Team self-management, organizational structure, and judgments of team effectiveness, *Journal of Managerial Issues* 16 (2) (2004) 248–265.
- [47] V. Vinekar, C.W. Slinkman, S. Nerur, Can agile and traditional systems development approaches coexist? an ambidextrous view, *Information Systems Management* 23 (3) (2006) 31–42.
- [48] R.K. Yin, *Case Study Research: Design and Methods*, Sage, Thousand Oaks, Calif., 2002.
- [49] C. Zannier, M. Chiasson, F. Maurer, A model of design decision making based on empirical results of interviews with software designers, *Information and Software Technology* 49 (6) (2007) 637–653.
- [50] N. Zazworka, K. Stapel, E. Knauss, F. Shull, V.R. Basili, K. Schneider, Are developers complying with the process: an XP study, in: *Proceedings of the 2010 ACM-IEEE International Symposium on Empirical Software Engineering and Measurement*, ACM, Bolzano-Bozen, Italy, 2010, pp. 1–10.